

Air University Islamabad Department of Computer Science



SUBMITTED 16 May, 2026

OOP Project. Mam Nimra Shafqat

Name: Imad Khan	ID: 2500623
------------------------	--------------------

TITLE:

BLOOD BANK MANAGEMENT SYSTEM

Submitted By:

Imad Khan (2500623), Mirza Saud (2500597), Umar Sarfaraz (2500624)

ABSTRACT

The *Blood Bank Management System* is a console-based application developed in C++ using Object-Oriented Programming concepts. The main purpose of this project is to manage donor and recipient information efficiently. The system stores blood donor records, recipient records, and processes blood requests based on blood group compatibility.

This project demonstrates important OOP concepts such as Classes, Objects, Encapsulation, Inheritance, Polymorphism, Abstraction, Constructors, Pointers, and Dynamic Memory Allocation. The system allows users to add donors, display donor information, add recipients, search donors by blood group, and process blood requests.

The project provides a simple and user-friendly solution for maintaining blood bank records and improving blood management operations.

TABLE OF CONTENTS

- Introduction
- Background of the Project
- Objectives of the Project
- Problem Statement
- Scope of the Project
- System Requirements
- OOP Concepts Used
- System Design
- Implementation Details
- Functional Modules
- Challenges Faced
- Calculations and Logic
- Advantages of the System
- Limitations of the System
- Future Enhancements
- Conclusion

INTRODUCTION

Blood donation is an important part of healthcare systems. Hospitals and blood banks require an efficient method to manage donor information and blood availability. Manual systems are slow, difficult to maintain, and may lead to errors.

The Blood Bank Management System is developed to automate the process of managing blood donors and recipients. The system helps users store donor details, search blood groups, and process blood requests easily.

This project is implemented using C++ and follows Object-Oriented Programming principles

BACKGROUND OF THE PROJECT

Traditional blood bank systems often rely on paper records or manual management methods. These systems are time-consuming and may create difficulties in finding donors during emergencies.

Modern software systems help improve efficiency, speed, and accuracy. Therefore, this project was designed to provide a computerized blood bank management solution using OOP concepts.

The project also serves as a practical implementation of programming concepts learned in the Object-Oriented Programming course.

OBJECTIVES OF THE PROJECT

The main objectives of the project are:

- To maintain donor records.
- To maintain recipient records.
- To search donors by blood group.
- To process blood requests efficiently.
- To apply Object-Oriented Programming concepts.
- To reduce manual record management.
- To improve data organization and accessibility.

PROBLEM STATEMENT

Managing blood donor and recipient information manually creates several problems such as:

- Difficulty in maintaining records.
- Delays in finding suitable donors.
- Risk of data loss.
- Time-consuming search process.
- Human errors in blood management.

The project solves these problems by providing an automated blood bank management

SCOPE OF THE PROJECT

The scope of this project includes:

- Adding donor information.
- Adding recipient information.
- Searching donors by blood group.
- Displaying donor and recipient records.
- Processing blood requests.
- Managing blood units.

The project is designed for educational purposes and small-scale blood bank management.

SYSTEM REQUIREMENTS

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB minimum
- Hard Disk: 100 MB free space

Software Requirements

- Operating System: Windows/Linux
- Programming Language: C++
- Compiler: Dev-C++, CodeBlocks, or Visual Studio
- Console Application Support

OOP CONCEPTS USED

1. Classes and Objects

Classes are used to create blueprints for donors and recipients.

Example:

- Person
- Donor
- Recipient

Objects are created from these classes.

2. Encapsulation

Data members are kept protected/private and accessed using member functions.

Example:

protected:
int id;
string name;

3. Inheritance

The Donor and Recipient classes inherit from the Person class.

Example:

```
class Donor : public Person
```

4. Polymorphism

Function overriding is used through virtual functions.

Example:

```
virtual void display() = 0;
```

5. Abstraction

The abstract class Person hides implementation details and provides a common structure.

6. Constructors

Constructors initialize object data automatically.

Example:

```
Donor(int i, string n, int a, string bg, int u)
```

7. Pointers

Pointers are used to store object addresses.

Example:

```
Donor* donors[10];
```

8. Dynamic Memory Allocation

Objects are created dynamically using new.

Example:

```
new Donor(id, name, age, bg, units);
```

SYSTEM DESIGN

Main Components

1. Person Class

- Base abstract class containing common data.

2. Donor Class

- Stores donor information and blood units.

3. Recipient Class

- Stores recipient details and blood requirements.

4. Main Menu

Controls all system operations.

SYSTEM FLOW

1. User selects menu option.
2. System receives input.
3. Data is stored in arrays using pointers.
4. Blood requests are processed.
5. Results are displayed.

IMPLEMENTATION DETAILS

The system is implemented using C++ programming language.

Features Implemented

- Add Donor
- Display Donors
- Add Recipient
- Display Recipients
- Search Donor by Blood Group
- Process Blood Requests

Sample Menu

1. Add Donor
2. Show Donors
3. Add Recipient
4. Show Recipients
5. Search Donor
6. Process Blood Request
0. Exit

FUNCTIONAL MODULES

Module 1: Donor Management

Functions:

- Add donor
- Display donor
- Search donor

Module 2: Recipient Management

Functions:

- Add recipient
- Display recipient

Module 3: Blood Request Processing

Checks:

- Blood group matching
- Available blood units

If blood is available, units are reduced from donor records.

CHALLENGES FACED

During project development, several challenges were faced:

- Understanding inheritance and polymorphism.
- Managing dynamic memory allocation.
- Handling user input correctly.
- Implementing menu-driven logic.
- Managing arrays of pointers.

CALCULATIONS AND LOGIC

Blood Request Logic

The system checks:

If donor blood group == requested blood group
AND donor units >= requested units

Then:

Available Units = Available Units - Requested Units

Example:

Donor Units = 10
Requested Units = 3

Remaining Units = 7

Search Logic

The system compares blood groups using loops.

Example:

```
if(donors[i]->getBloodGroup() == bg)
```

ADVANTAGES OF THE SYSTEM

- Easy record management
- Faster donor search
- Reduced manual work
- Better organization
- Simple user interface
- Demonstrates OOP concepts effectively

LIMITATIONS OF THE SYSTEM

- Console-based interface only
- Limited storage capacity
- No database integration
- No login/security system

FUTURE ENHANCEMENTS

Future improvements may include:

- Graphical User Interface (GUI)
- Database connectivity
- Online donor registration
- Admin login system
- Blood compatibility checking
- File handling for permanent storage

CONCLUSION

The Blood Bank Management System is a useful and efficient software application developed using the C++ programming language and Object-Oriented Programming principles. The main purpose of this project is to manage blood donor and recipient records in an organized and systematic way. The system successfully performs important operations such as adding donor information, storing recipient details, searching donors by blood group, displaying records, and processing blood requests.

This project demonstrates the practical implementation of major OOP concepts including Classes, Objects, Encapsulation, Inheritance, Polymorphism, Abstraction, Constructors, Pointers, and Dynamic Memory Allocation. These concepts helped in creating a modular, reusable, and well-structured program. The use of inheritance reduced code duplication, while polymorphism and abstraction improved flexibility and maintainability of the system.

The project also helped in understanding how real-world problems can be solved through programming and software development techniques. During the development process, various programming challenges were faced such as handling dynamic memory, managing arrays of pointers, implementing menu-driven operations, and designing proper class relationships. Solving these problems improved logical thinking, debugging skills, and coding practices.

The Blood Bank Management System provides a simple and user-friendly solution for small-scale blood bank operations. Although the current version is console-based and designed mainly for educational purposes, it can be further improved by adding database connectivity, graphical user interface (GUI), login authentication, file handling, and online donor registration features.

Overall, this project enhanced practical knowledge of Object-Oriented Programming and provided valuable experience in designing and implementing a complete software application. The project successfully achieved its objectives and proved to be an effective learning experience in software development and system design.

SOURCE CODE

```
#include <iostream>
#include <string>
using namespace std;

/*
=====
BLOOD BANK MANAGEMENT SYSTEM
OOP Concepts Included:
1. Classes & Objects
2. Encapsulation
3. Inheritance
4. Polymorphism
5. Abstraction
6. Constructors
7. Pointers
8. Dynamic Memory
=====
*/

// === ABSTRACT CLASS ===
class Person {
protected:
    int id;
    string name;
    int age;

public:
    // Constructor
    Person(int i, string n, int a) {
        id = i;
        name = n;
        age = a;
    }

    // Pure Virtual Function (Abstraction)
    virtual void display() = 0;

    // Getter Functions (Encapsulation)
    int getId() {
        return id;
    }

    string getName() {
        return name;
    }
}
```

```

    }
};

// === DONOR CLASS ===
class Donor : public Person { // Inheritance
private:
    string bloodGroup;
    int units;

public:
    // Constructor
    Donor(int i, string n, int a,
          string bg, int u)
        : Person(i, n, a) {

        bloodGroup = bg;
        units = u;
    }

    // Function Overriding (Polymorphism)
    void display() override {

        cout << "\n---- DONOR ----";
        cout << "\nID: " << id;
        cout << "\nName: " << name;
        cout << "\nAge: " << age;
        cout << "\nBlood Group: " << bloodGroup;
        cout << "\nUnits: " << units << endl;
    }

    string getBloodGroup() {
        return bloodGroup;
    }

    int getUnits() {
        return units;
    }

    void reduceUnits(int u) {
        units = units - u;
    }
};

// === RECIPIENT CLASS ===
class Recipient : public Person { // Inheritance

```

```

private:
    string bloodNeeded;
    int unitsNeeded;

public:
    // Constructor
    Recipient(int i, string n, int a,
              string bg, int u)
        : Person(i, n, a) {

        bloodNeeded = bg;
        unitsNeeded = u;
    }

    // Polymorphism
    void display() override {

        cout << "\n----- RECIPIENT -----";
        cout << "\nID: " << id;
        cout << "\nName: " << name;
        cout << "\nAge: " << age;
        cout << "\nBlood Needed: " << bloodNeeded;
        cout << "\nUnits Needed: " << unitsNeeded << endl;
    }

    string getBloodNeeded() {
        return bloodNeeded;
    }

    int getUnitsNeeded() {
        return unitsNeeded;
    }
};

// ==== MAIN FUNCTION ====
int main() {

    // Pointers + Dynamic Memory
    Donor* donors[10];
    Recipient* recipients[10];

    int donorCount = 0;
    int recipientCount = 0;

    int choice;

```

```

do {

    cout << "\n\n===== BLOOD BANK MENU =====";
    cout << "\n1. Add Donor";
    cout << "\n2. Show Donors";
    cout << "\n3. Add Recipient";
    cout << "\n4. Show Recipients";
    cout << "\n5. Search Donor by Blood Group";
    cout << "\n6. Process Blood Request";
    cout << "\n0. Exit";

    cout << "\nEnter Choice: ";
    cin >> choice;

    switch(choice) {

// ===== ADD DONOR =====
case 1: {

        int id, age, units;
        string name, bg;

        cout << "Enter Donor ID: ";
        cin >> id;

        cin.ignore();

        cout << "Enter Name: ";
        getline(cin, name);

        cout << "Enter Age: ";
        cin >> age;

        cin.ignore();

        cout << "Enter Blood Group: ";
        getline(cin, bg);

        cout << "Enter Units: ";
        cin >> units;

        // Dynamic Memory Allocation
        donors[donorCount] =
            new Donor(id, name, age, bg, units);

```

```
        donorCount++;

        cout << "Donor Added Successfully!\n";

        break;
    }

    // ===== SHOW DONORS =====
    case 2: {

        if(donorCount == 0) {
            cout << "No Donors Found.\n";
        }

        for(int i=0; i<donorCount; i++) {

            // Runtime Polymorphism
            donors[i]->display();
        }

        break;
    }

    // ===== ADD RECIPIENT =====
    case 3: {

        int id, age, units;
        string name, bg;

        cout << "Enter Recipient ID: ";
        cin >> id;

        cin.ignore();

        cout << "Enter Name: ";
        getline(cin, name);

        cout << "Enter Age: ";
        cin >> age;

        cin.ignore();

        cout << "Enter Blood Needed: ";
        getline(cin, bg);
```

```

        cout << "Enter Units Needed: ";
        cin >> units;

        recipients[recipientCount] =
            new Recipient(id, name, age, bg, units);

        recipientCount++;

        cout << "Recipient Added Successfully!\n";

        break;
    }

    // ==== SHOW RECIPIENTS ====
    case 4: {

        if(recipientCount == 0) {
            cout << "No Recipients Found.\n";
        }

        for(int i=0; i<recipientCount; i++) {

            recipients[i]->display();
        }

        break;
    }

    // ==== SEARCH DONOR ====
    case 5: {

        string bg;
        bool found = false;

        cin.ignore();

        cout << "Enter Blood Group: ";
        getline(cin, bg);

        for(int i=0; i<donorCount; i++) {

            if(donors[i]->getBloodGroup() == bg) {

                donors[i]->display();
            }
        }
    }
}

```

```

        found = true;
    }
}

if(!found) {
    cout << "No Donor Found.\n";
}

break;
}

// ==== PROCESS BLOOD REQUEST ====
case 6: {

    string bg;
    int units;
    bool found = false;

    cin.ignore();

    cout << "Enter Blood Group Needed: ";
    getline(cin, bg);

    cout << "Enter Units Needed: ";
    cin >> units;

    for(int i=0; i<donorCount; i++) {

        if(donors[i]->getBloodGroup() == bg &&
           donors[i]->getUnits() >= units) {

            donors[i]->reduceUnits(units);

            cout << "Blood Request Completed!\n";

            found = true;
            break;
        }
    }

    if(!found) {
        cout << "Blood Not Available.\n";
    }

    break;
}

```

```

    }

    // ===== EXIT =====
    case 0:
        cout << "Program Ended.\n";
        break;

    default:
        cout << "Invalid Choice!\n";
    }

} while(choice != 0);

// ===== MEMORY DEALLOCATION =====
for(int i=0; i<donorCount; i++) {
    delete donors[i];
}

for(int i=0; i<recipientCount; i++) {
    delete recipients[i];
}

return 0;
}

```

OUTPUT

Menu

Output	Clear
<pre> ===== BLOOD BANK MENU ===== 1. Add Donor 2. Show Donors 3. Add Recipient 4. Show Recipients 5. Search Donor by Blood Group 6. Process Blood Request 0. Exit </pre>	

Enter 1:

- For Add Donor

```
===== BLOOD BANK MENU =====
1. Add Donor
2. Show Donors
3. Add Recipient
4. Show Recipients
5. Search Donor by Blood Group
6. Process Blood Request
0. Exit
Enter Choice: 1
Enter Donor ID: 2500623
Enter Name: Imad Khan
Enter Age: 19
Enter Blood Group: A+
Enter Units: 1
Donor Added Successfully!
```

Enter 2:

- For Show Donors

```
===== BLOOD BANK MENU =====
1. Add Donor
2. Show Donors
3. Add Recipient
4. Show Recipients
5. Search Donor by Blood Group
6. Process Blood Request
0. Exit
Enter Choice: 2

----- DONOR -----
ID: 2500623
Name: Imad Khan
Age: 19
Blood Group: A+
Units: 1
```

Enter 3:

- For Add Recipient

```
===== BLOOD BANK MENU =====
1. Add Donor
2. Show Donors
3. Add Recipient
4. Show Recipients
5. Search Donor by Blood Group
6. Process Blood Request
0. Exit
Enter Choice: 3
Enter Recipient ID: 2500597
Enter Name: Mirza Saud
Enter Age: 19
Enter Blood Needed: A+
Enter Units Needed: 1
Recipient Added Successfully!
```

Enter 4:

- For Show Recipients

```
===== BLOOD BANK MENU =====
1. Add Donor
2. Show Donors
3. Add Recipient
4. Show Recipients
5. Search Donor by Blood Group
6. Process Blood Request
0. Exit
Enter Choice: 4

----- RECIPIENT -----
ID: 2500597
Name: Mirza Saud
Age: 19
Blood Needed: A+
Units Needed: 1
```

Enter 5:

- For Search Donor by Blood Group

```
===== BLOOD BANK MENU =====
1. Add Donor
2. Show Donors
3. Add Recipient
4. Show Recipients
5. Search Donor by Blood Group
6. Process Blood Request
0. Exit
Enter Choice: 5
Enter Blood Group: A+

----- DONOR -----
ID: 2500623
Name: Imad Khan
Age: 19
Blood Group: A+
Units: 1
```

Enter 6:

- For Process Blood Request

```
===== BLOOD BANK MENU =====
1. Add Donor
2. Show Donors
3. Add Recipient
4. Show Recipients
5. Search Donor by Blood Group
6. Process Blood Request
0. Exit
Enter Choice: 6
Enter Blood Group Needed: A+
Enter Units Needed: 1
Blood Request Completed!
```

Enter 0:

- For Exit

```
===== BLOOD BANK MENU =====  
1. Add Donor  
2. Show Donors  
3. Add Recipient  
4. Show Recipients  
5. Search Donor by Blood Group  
6. Process Blood Request  
0. Exit  
Enter Choice: 0  
Program Ended.
```

---END---